Systems Administration (ASIST)

Practical Theoretical Class 06

Network File Systems (NAS) - SMB/CIFS and NFS
Network packet filtering – *Firewall*
Lightweight Directory Access Protocol (LDAP)
Linux – OpenLDAP

NAS protocols – SMB/CIFS

Among other highlights are two network file system protocols: **SMB/CIFS** and **NFS**, respectively originating from Windows and Unix systems. Currently either Windows servers or Linux servers can use both protocols.

SMB/CIFS (*Server Message Block/Common Internet File System*) refers to a protocol developed over time by Microsoft that allows file system sharing across the network. Effectively the protocol is **SMB**, **CIFS** refers to the latest dialect supported by **SMB**.

SMB has a dialect negotiation mechanism, which allows it to support Windows clients and servers of different versions, in which negotiation both players determine which is the latest dialect that is supported by both. In SMB dialects are identified by names, the oldest is **PC NETWORK PROGRAM 1.0**, the latest is **DOS LANMAN 2.1** and **NT LM 0.12**, the latter is CIFS.

Access to SMB/CIFS file systems is user session driven, the user authenticates to the server and obtains a share list provided by the server for that user. In addition to file system folders, printers can also be shared. Access to shares is conditioned by permissions (ACLs) according to the authenticating user.

Of course in the context of a Windows domain, after logging in to the domain, the user is immediately authenticated to all servers that are members of the domain.

Folder shares via SMB/CIFS are identified using the *Universal Naming Convention* (**UNC**) format:

\\{SERVER-NAME}\{SHARE-NAME}

Where **{SERVER-NAME}** represents the server providing the share, which can be a NetBIOS simple name, a DNS name, or an IP address. **{SHARE-NAME}** is the name given to the share on the server, by default the folders are shared using the folder name, however that name can be changed and / or multiple shares can be set for the same folder, each with a different name.

Each share has an associated ACL that defines access permissions for users and groups. Share ACLs only have three permissions: **Full Control**, **Change** and **Read**, however permissions set for sharing do not override permissions on NTFS file system.

By default when creating a share, the Read permission is given to the implicit group **Everyone**, but this does not mean that all users have access, it all depends on the NTFS permissions set on the folder being shared. It's really common practice to give the **Everyone** group **Full Control** permission on shares, so all control is done by NTFS ACLs.

Windows domains - *home* and *profile* sharing

Windows user accounts set ***Profile Path*** and ***Home Folder*** attributes. In the context of a domain these attributes must match network shares made available by domain servers, only then will they be accessible on all workstations that log in to the domain.

Users' home folders should be placed in a folder shared by one of the domain's servers, for example \\SERVER\HOMES. The ***Home Folder*** attribute of each user account can then take the form **\\SERVER\HOMES\{username}**, where {**username**} is the username (assuming each folder has a username equal to the username).

The share **\\SERVER\HOMES** can be set to **Full Control** permission for **Everyone**, the effective permissions will be dictated by the ACLs of the NTFS.

For ***Roaming Profile*** (***Profile Path***), identical strategy must be implemented, possibly using another domain server.

In a simpler configuration a ***Home Folder*** subfolder can be used for ***Profile Path***, for example:

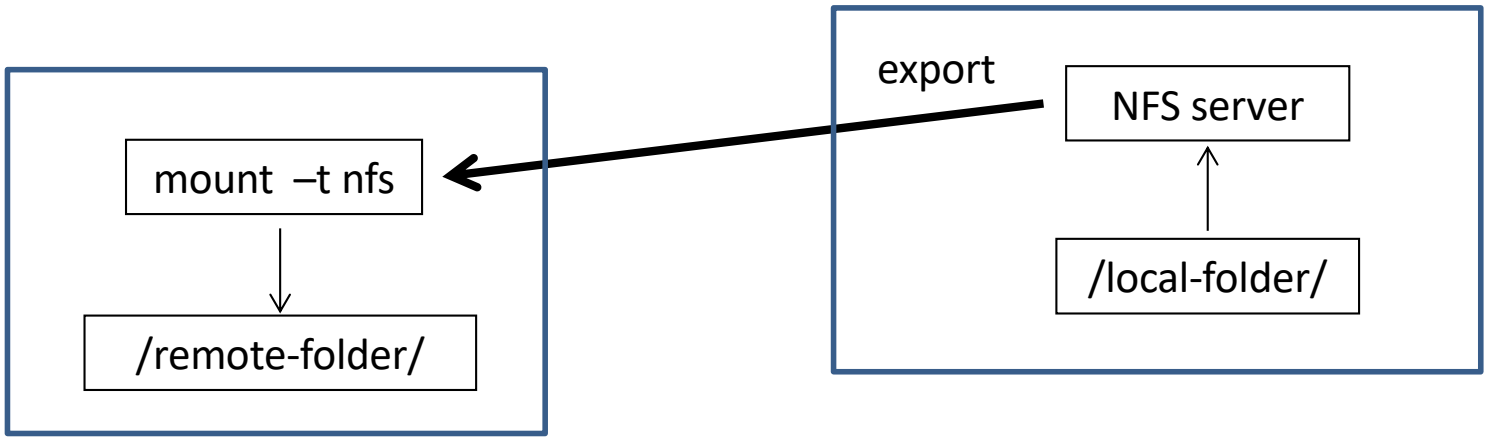
\\SERVER\HOMES\{username}\Profile\{username}

NAS protocols – NFS (Network File System)

The NFS protocol originates from Unix systems, was initially developed by *Sun Microsystems*. Traditional NFSv2 and NFSv3 implementations are fully intended to be handled exclusively by system administrators, no user authentication is supported.

NFSv4 supports new forms of authentication, including individual user authentication.

Focusing on system administrators' use, NFS lets you **export local folders** from a machine that acts as an NFS server (the export verb is reasonably equivalent to sharing). A folder exported by NFS can be **mounted by another machine** (NFS client) and thus integrated into the file system and available to applications and users.



Unix – NFSv2 and NFSv3 – NFS Server

On Unix systems the NFS server is configured through the file **/etc/exports**, this file defines the local folders that are exported and under what conditions.

In NFSv2 and NFSv3, access to exported folders is controlled through the client's IP address, so in the **/etc/exports** file, for each exported folder the accepted client IP addresses are listed. The IP address can be specified in many ways, including an IP address, an IP network (network address and prefix), a name that is resolved to an IP address and by metacharacters, for example ***** matches any address..

For each export various options can be set including read only **(ro)**, read and write **(rw)**. By default exports always use the **root_squash** option, this means that the **root** user of the client machine will not have the same permissions on the exported file system as the **root** user of the server machine has.

The remaining users will have exactly the same permissions on both the server and the client, since the UID are the same.

Example of **/etc/exports** file:

```
/users/home       190.130.60.0/255.255.255.0(ro)
/users/backups    vsrv2(rw)  *(ro)
/usr/local/dump   190.130.60.10(rw)    190.130.60.123(rw)
```

Unix – NFSv2 and NFSv3 – NFS client

Authorized NFS clients can mount file systems exported by the server using the **mount** command:

mount –t nfs {SERVER}:{FOLDER} {MOUNT-POINT}

Where **{SERVER}** represents the IP address or DNS name of the NFS server, **{FOLDER}** represents the name of the exported folder (as in the server's /etc/exports file) and **{MOUNT-POINT}** represents an empty local folder where the remote folder will be made available.

To make the configuration permanent, the mount must be added to the **/etc/fstab** file to be done at machine startup.

All file system attributes are exported, including UID (owner), GID, and permissions associated with each object.

As a result, the client machine users' permissions on the mounted file system are exactly the same as they have on the server, it is only necessary to ensure that the UID and GID of each user and group is exactly the same on all machines.

Ensuring that UID and GID are the same on all machines is easily achieved if all machines use the same centralized user and group account repository, for example LDAP.

Authentication by IP address – NFSv2 and NFSv3

Authentication by IP address can be considered safe if applied with some caution:

- **There must be full trust in the client node administrator.** Because reserved port numbers (<1024) are used, both client and server know that they are dialoging with an application under the control of the node administrator. Particularly when the server receives a request from a client it verifies that it comes from an authorized IP address and a reserved port number.

- **There must be confidence in how the network works.** The system is totally vulnerable to *IP Spoofing* attacks, so it should only be used on a network where users do not have access to, typically a network where there are only servers, a DMZ (*DeMilitarized zone*).

- **There must be confidence in how name resolution works.** If machine names were used instead of addresses, the system could be attacked through DNS. If the DNS server is compromised it may give false answers and thus cause the NFS server to allow the wrong addresses. As a safety precaution, in the context of NFS, it is advisable to always use IP addresses and not DNS hostnames.

Using NFS for *homes* sharing

In an environment where there is a set of Linux servers, using a centralized user and group account repository, for example, an LDAP-accessible database, allows users and groups to be the same on all servers.

As a result users can login to either server, NFSv2/NFSv3 can then be used to equally make ***home directories*** available on all servers.

Home directories will be housed in a folder on a server that is exported by NFS to all servers where home directories are needed. Of course the folder containing the home directories should be exported in ***read-write***. CoBecause NFS maintains all object properties, including associated UID/GID and permissions, users will have the correct permissions on their *home directory* on either server.

On servers receiving the exported folder, care must be taken to mount the exported folder in a location that allows the *home directory* attribute of the user account to match the actual folder location.

Eventually if they do not match it will also be simple to solve the problem through a symbolic link.

Packet filtering - firewall

Firewalls are mechanisms integrated into network nodes that are intended to condition the passage of network traffic.

An intermediate node such as a ***router*** allows you to control the traffic that is transferred between different networks. At an end node such as a server or workstation, it allows you to condition the traffic that enters (***inbound***) or leaves (***outbound***) that node.

In the case of an end node, all attention is focused on ***inbound*** traffic as threats are assumed to come from the network rather than from within the node.

There are several ways to analyze network traffic corresponding to different firewall type designations:

- Individual analysis of each package (designated ***stateless***).
- Analysis of complete transactions at the level of ***connection-oriented*** transport protocols and application protocols (designated ***stateful***).

Windows Server - Windows Firewall with Advanced Security

Stateless rules are divided into ***Inbound Rules*** and ***Outbound Rules***, respectively applied to traffic coming from the network and traffic that the node intends to emit.

Packets arriving from the network or intended to be sent to the network are confronted with the rule lists, respectively *Inbound* and *Outbound*.

Each rule defines criteria regarding which packets correspond to it and the action to be taken in case of a match: ***Allow***, ***Block*** and ***Allow if secure***. In the latter case, the criteria to consider ***secure*** can be defined for each rule.

Inbound traffic is only allowed if it matches an ***Allow*** rule and does not match any ***Block*** rules (in ***Inbound Rules***).

Outbound traffic is blocked only if it matches a ***Block*** rule (in ***Outbound Rules***).

As can be seen the application of ***inbound*** rules is much more restrictive of the application of ***outbound*** rules.

Inbound, if there is no match the packet is **blocked**. **Outbound**, if there is no match the packet is **allowed**.

For each rule the criteria for matching packages are defined, they may be based on:

- **Programs and Services** - local applications to which the packages are intended (*inbound*) or where the packages originate (*outbound*).
- **Protocols and Ports** – protocols (TCP, UDP, ICMPv4,…) and, where applicable, source and destination port numbers (TCP and UDP) or ICMP message type.
- **Scope** – packet source and destination node addresses (IPv4 and / or IPv6).
- **Local Principals** – local users (who owns the applications that will receive the package or have issued the package).
- **Remote Users** – remote users (of *Active Directory*).
- **Remote Computers** – remote systems (of *Active Directory*).

It is also possible to condition the application of the rules to the network context in which the machine is located (***Profile***):

- **Domain** – if the machine is connected to a network that belongs to the domain.
- **Private** – if the machine is connected to a network that has been classified as private.
- **Public** – if the machine is connected to a network that has been classified as public.

Linux – Netfilter and iptables commands

Netfilter is a traffic control system integrated in the Linux kernel. It allows, among other features, to define rules regarding which network packages are allowed.

To manage Netfilter the administrator uses the iptables and ip6tables commands, respectively for IPv4 or IPv6 rules.

Through iptables 5 tables can be managed: **filter**, **nat**, **mangle**, **raw** and **security**.

The **filter** table is used to define packet permission (**ACCEPT**) or non-permission (**DROP**) criteria. The **nat** table is used to define ***Network Address Translation***, Source (SNAT), and Destination (DNAT), handling rules.

When using the **iptables** command the table to be manipulate can be indicated with the **-t** argument, by default the **filter** table is assumed.

In each table there are ***chains*** of rules. Each table has some predefined chains (*built-in*), but others chains can be created. Each predefined chain applies to packets in certain situations.

In each chain, rules are numbered starting from 1 (first rule). Each packet applicable to the chain is sequentially confronted with each rule in the chain.

Table	Built-in chains	Objectives
filter	INPUT	ACCEPT / DROP rule sequence applied to packets when they arrive from the network and are destined for the node.
	FORWARD	ACCEPT / DROP rule sequence applied to packets when they arrive from the network but are not intended for the node. Used when the node operates as a **router**.
	OUTPUT	ACCEPT / DROP rule sequence applied to packets to be issued by node and originating from node.
nat	PREROUTING	Address changes to apply to packets arriving at the node (either non-node) before the packet is forwarded.
	OUTPUT	Address changes to apply to packets issued by node (originating from node) before forwarding.
	POSTROUTING	Address changes to apply to packets that will be issued by the node (whether or not they originate from the node) after the packet is forwarded.

Each rule in a chain defines packet **matching criteria** and the **action to take** (**target**) in case of matching. Not matching the rule the packet will be confronted with the next chain rule.

If a packet reaches the end of a chain, the action defined in the **chain policy** will be performed on it (only for *built-in* chains).

iptables - targets

Each rule defines the action to take in case of matching a package through the options **-j target** or **-g chain.**

Action	Description of action over packet
-j ACCEPT	Pass the packet, terminates processing for the packet chain.
-j DROP	Block (destroy) the packet, finish chain processing for the packet.
-g {chain}	Processes the chain identified by {chain}. In addition to the *built-in* it is possible to create new rule chains. Using the **–g** option the packet will be confronted with the designated chain rules.
-j RETURN	Ends processing of the current chain and returns to the source chain to continue processing of the following rules. If the current chain is *built-in*, the chain policy applies to the package.
-j SNAT	Change of source address (IP and/or port) (via option **--to-source**). Only valid in the **nat** table in **POSTROUTING** chain. Ends chain processing for the packet.
-j DNAT	Change of destination address (IP and/or port) (via option **--to-destination**). Only valid in the **nat** table in **PREROUTING** and **OUTPUT** chains. Ends chain processing for the packet.
-j LOG	Log packet to system logs. Does not end chain processing, continues processing the packet through the next chain rule.

iptables - matching criteria

Each rule defines packet **matching criteria**, the criteria are specified through options. If multiple criteria are defined, **the rule will only apply to packets for which all criteria are met**. Criteria may be negated if the option is preceded by the symbol **!**.

Criteria	Criterion description
-i {iface}	Packet was received via network interface {iface}.
-o {iface}	The packet will be issued through the network interface {iface}.
-p {proto}	The packet carries data from the designated protocol {proto}. The value of {proto} can be for example **tcp**, **udp**, **icmp** or **icmpv6**. The value **all** represents any protocol, is equivalent to not defining this criterion.
-s {address}[/{mask}]	The source address of the packet is the specified addresses, if omitted /{mask} the address is interpreted as a single node, ie mask 32 or 128 bits respectively for IPv4 and IPv6.
-d {address}[/{mask}]	Identical to –s option, but specifying packet destination address.
--dport {port}[:port2]	The packet's destination port number is {port}, or is within the range [{port}, {port2}] (only valid for **–p udp** or **–p tcp**).
--sport {port}[:port2]	The packet's source port number is {port}, or is within the range [{port}, {port2}] (only valid for **–p udp** or **–p tcp**).

iptables – definition of rule chains

Command	Command description
-N {chain}	Create a new chain.
-X {chain}	Deletes a previously defined chain (not applicable for built-in chains).
-F [{chain}]	Delete all rules from chain, if chain is omitted delete all rules from all table chains.
-P {built-in-chain} {target}	Sets chain policy, valid only for built-in chains, valid targets are ACCEPT and DROP.
-A {chain} {rulespec}	Appends a rule (append) to the designated string. {rulespec} is a rule definition including criteria and target.
-I {chain} [{num}] {rulespec}	Inserts into the chain a new rule at line {num}, if {num} is not specified inserts at the first position.
-R {chain} {num} {rulespec}	Replaces the rule at position {num}. Of the chain.
-D {chain} {num}	Deletes the rule at position {num} of the chain.
-L [{chain}]	Displays the list of rules in the designated string or all table strings. The **–n** option displays all addresses and ports in numeric form. The **–v** option displays more details. Option **--line-numbers** displays the numbering of the rules.

iptables - modules

Option **-m** allows the loading of modules to support additional funcionality. For exemple, **state** (**-m state**) module allows definition of matching criteria **--state {states}** where *states* is a comma-separated list of values such as: **ESTABLISHED**, **NEW** and **RELATED**. The **ESTABLISHED** criterion applies to TCP traffic on already established connections, **NEW** to new TCP connections. The **RELATED** criterion applies to response UDP traffic, ie packets that correspond to previously made UDP request responses.

Linux – *routing*

By default, the Linux *kernel* rejects incoming packets not addressed to the host (does not forward traffic between networks). This behavior can be changed by modifying the **net.ipv4.ip_forward** and **net.ipv6.conf.all.forwarding** system variables to the value **1**, respectively for IPv4 and IPv6. The node will act as a router.

To make these settings permanent (applied whenever the system starts) they must be set in the configuration file **/etc/sysctl.conf**.

LDAP (*Lightweight Directory Access Protocol*)

The LDAP protocol is designed to allow expedited access to information services across the network, typically queries and information retrieval.

From an LDAP point of view information is stored in objects and organized in a tree structure (***DIT - Directory Information Tree***). Each object is the instantiation of a class, objects contain attributes where the data itself is stored.

The class(***LDAP objectClass***) defines the attributes of objects such as:

- What are the attributes and their name.

- What kind of data an attribute can store (what values can it assume).

- If the attribute is mandatory or optional.

- If the attribute can be repeated several times in the object or must be unique.

Object classes are normalized and should not be defined arbitrarily, classes are defined through ***schema***.

There are a wide variety of classes available for a wide range of applications.

LDAP - classes and objects

Class definition (in *schema*) uses inheritance, ie you can define a new class from an existing class and add new attributes. There are 3 types of class:

ABSTRACT – is a top class defined with the goal of being homestead by concrete classes. It is not possible to create objects with this kind of classes.

STRUCTURAL – is a class with which objects can be created, an object must belong to a structural class, but cannot belong to more than one.

AUXILIARY – is a class for the purpose of adding attributes to an object that already belongs to a structural class.

An object must belong to one and only one structural class, but can belong to several auxiliary classes.

Example of part of a *schema*:

```
objectclass ( 2.5.6.2 NAME 'country' SUP top STRUCTURAL
  DESC '2 character iso 3611 assigned country code'
  MUST c
  MAY ( searchGuide $ description ) )
```

LDAP – DN (*Distinguished Name*) and root DN

Each object in the DIT is uniquely identified by DN. Within the branch of the DIT where an object is located is uniquely identified by the value of one of its attributes this is the *Relative Distinguished Name* (**RDN**) of the object. The DN consists of the RDN followed by the branch DN (parent object DN). The various elements of DN and RDN are separated by a comma.

The top of the hierarchy (DIT) root itself has a DN that is defined by the administrator (**root-DN**). Any object in a DIT will necessarily include DIT root-DN in its DN.

According to current standards, **root-DN** must match the DNS name of the local domain separated into components by the *Domain Component* (**dc**) attribute:

dc=sub-domain,dc=sub-domain,…,dc=top-domain

For example for the DNS domain *dei.isep.ipp.pt*, the **root-DN** of the DIT should be: **dc=dei,dc=isep,dc=ipp,dc=pt**

The object with **dn**: **cn=user1,ou=users,dc=dei,dc=isep,dc=ipp,dc=pt** is perfectly identified, has RDN **cn=user1** and is located in the branch **ou=users,dc=dei,dc=isep,dc=ipp,dc=pt**.

Linux –OpenLDAP server (http://www.openldap.org)

One of the most commonly used LDAP servers on Linux is OpenLDAP. Traditionally configuration is done through the **slapd.conf** file, but in recent distributions like current Ubuntu the configuration is defined in a server DIT with root-DN **cn=config**.

Some of the base configuration elements are:

- The **schemas** to support (object classes).
- **Backend** – the database where objects are stored, the most common is **bdb** (*Oracle Berkeley DB*), but can also use **SQL**.
- **Suffix** – refers to DIT root-DN.
- **Admin Account** – identifies an object in the DIT that will correspond to the LDAP administrator, the object must contain the **userPassword** attribute.
- **Attributes to índex** - To speed up the tree search, the most relevant attributes should be indexed, since the **objectClass** attribute. For user accounts other attributes like **cn**, **uid**, **uidNumber** and **gidNumber** should also be indexed.

Once defined the basic settings, account management can be used to manage objects from the root set-DN and, if applicable, to manage the configuration resident in the DIT **cn=config**.

LINUX - LDAP – LDIF files

Once the administration account is established, LDAP can then be used to manage the objects in the DIT, for this purpose the following commands can be used:

ldapadd – add an object.

ldapdelete – delete an object.

ldapmodify – add, delete or change an object (attributes), including change the name (**DN**). Note that changing the DN may include moving the object from one branch to another (**moddn**) or not (**modrdn**).

These commands accept as arguments files in *LDAP Data Interchange Format* (**LDIF**) format with instructions for actions to be performed using a specific syntax, for example:

```
dn: cn=user1,dc=example,dc=com
        changetype: modify
        replace: mail
        mail: modme@example.com
        -
        add: title
        title: Grand Poobah
        -
```

If a file with this content is provided to the **ldapmodify** command it would change the content of the ***mail*** attribute and append the ***title*** attribute to the object with DN **cn=user1,dc=example,dc=com.**
Each action defined must end with a line consisting of the symbol **-.**